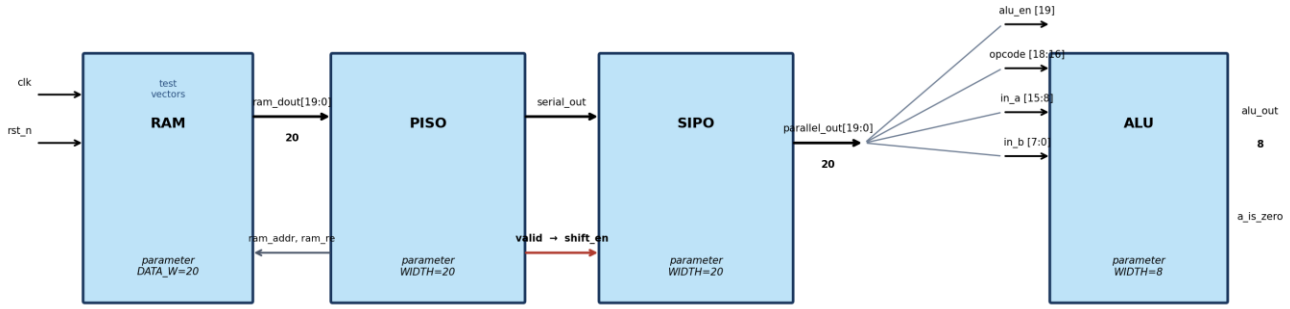
Lab 4 RAM → PISO → SIPO → ALU Data Path

Objective: To use Verilog sequential and combinational constructs to build a small streaming data path: a RAM of pre-stored 20-bit test vectors is serialized by a parallel-in serial-out (PISO) shift register, reassembled by a serial-in parallel-out (SIPO) shift register, and finally decoded and processed by a parameterized-width ALU.

This lab extends the RAM/PISO front end onto the SIPO + ALU design you already built. Test vectors are no longer applied directly to the SIPO's serial_in port by a testbench; instead they are stored in RAM, read out one word at a time, and streamed out serially by a PISO register. The PISO's valid output tells the downstream SIPO exactly when serial data is present, so valid is wired directly to the SIPO's shift_en (enable) input. This introduces students to a simple, self-timed handshake between two shift registers, and to reading parameterized-width memories in Verilog.

Block Diagram



Part A — RAM

Design a module named ram that stores a set of 20-bit test vectors and presents them for serialization by the PISO.

Port	Width	Description
clk	1	System clock.
rst_n	1	Active-low asynchronous reset.
wr_en	1	Write enable. When 1, din is written to mem[waddr] on the next rising edge of clk.
addr	8	Write address.
din	20	Write data (used only for optional testbench loading; see note below).
rd_en	1	Read enable, driven by PISO. When 1, dout is updated from mem[raddr] on the next rising edge of clk.
dout	20	Registered read data, presented to PISO one cycle after re and raddr are asserted.
valid	1	Asserted when ram extract output; low otherwise

Design requirements:

- Parameterize the address width (default ADDR_WIDTH =8,) and the data width (default DATA_WIDTH = 20).
- dout is registered: it updates one clock cycle after rd_en and addr are presented, and holds its value until the next read.

Part B — PISO (Parallel-In Serial-Out)

Design a module named piso_reg that reads successive words from RAM and serializes each one onto serial_out, MSB first.

Port	Width	Description
clk	1	System clock.
Rst_n	1	Active-low asynchronous reset.
Parallel_in	20	Parallel word read back from RAM.
en	1	enable issued to RAM.
serial_out	1	Serialized data bit, MSB-first, one new bit per clock while valid = 1.
valid	1	Asserted for exactly WIDTH clock cycles while serial_out carries the bits of the current word; low otherwise.

Design requirements:

- Parameterize the register width (default WIDTH = 20) and the RAM address width, matching the RAM module.
- After reset, PISO issues en = 1 to fetch the Data.
- Once en is valid, PISO parallel-loads it into an internal shift register and, on each subsequent clock edge, shifts it left by one bit, driving the outgoing bit onto serial_out (LSB first) and asserting valid = 1 for exactly WIDTH cycles.
- serial_out and valid must be 0 whenever a word is not actively being shifted out (e.g., during the one-cycle RAM fetch latency between words).

Part C — SIPO (Serial-In Parallel-Out)

Design a module named sipo_reg that reassembles the 20-bit word from PISO's serial stream.

Port	Width	Description
clk	1	System clock. Sampling/shifting occurs on the rising edge.
rst_n	1	Active-low asynchronous reset. Clears parallel_out to 0.
shift_en	1	Shift enable, driven by PISO's valid signal. When 1, serial_in is shifted in on the next clock edge.
serial_in	1	Serial data input, MSB-first, driven by PISO's serial_out.
parallel_out	20	Parallel output register; holds the last 20 bits received.

Design requirements:

- Parameterize the register width (default WIDTH = 20) so the module can be reused for other widths.
- On each rising edge of clk, if shift_en = 1, shift parallel_out left by one bit and load the incoming serial_in into bit [0] (i.e., data arrives LSB first).
- rst_n is asynchronous and active-low; when asserted (0), parallel_out is cleared to all zeros regardless of clk.
- Because shift_en is driven directly by PISO's valid signal, parallel_out completes one full 20-bit word exactly WIDTH cycles after valid first goes high — i.e., in the same cycles that PISO is shifting a word out.

Bit-Field Assignment

Once fully loaded, parallel_out[19:0] is interpreted as follows:

Signal	Bit Range	Width	Description
alu_en	parallel_out[19]	1	ALU enable (MSB of the shift register output)
opcode	parallel_out[18:16]	3	ALU operation select
in_a	parallel_out[15:8]	8	ALU operand A
in_b	parallel_out[7:0]	8	ALU operand B

Part D — The ALU

Design a module named alu (you may reuse and extend your previous design) that accepts the fields produced by the SIPO.

Port	Width	Description
in_a	8	Operand A, taken from parallel_out[15:8]
in_b	8	Operand B, taken from parallel_out[7:0]
opcode	3	Operation select, taken from parallel_out[18:16]
alu_en	1	Enable, taken from parallel_out[19] (the MSB)
alu_out	8	Result output
a_is_zero	1	Asynchronous flag: 1 when in_a == 0, else 0

Specifications:

- Parameterize the ALU operand/result width (default WIDTH = 8) and assign it a default value.
- a_is_zero is combinational: it is 1 whenever in_a = 0, and 0 otherwise, regardless of alu_en or opcode.
- When alu_en = 0, the ALU is disabled: force alu_out = 0.
- When alu_en = 1, alu_out is selected by opcode according to the table below.

Opcode [2:0]	Instruction	Operation	alu_out
000	ADD	Addition	in_a + in_b
001	SUB	Subtraction	in_a - in_b
010	AND	Bitwise AND	in_a & in_b
011	XOR	Bitwise XOR	in_a ^ in_b

Opcode [2:0]	Instruction	Operation	alu_out
100	OR	Bitwise OR	in_a | in_b
101	OUT A	Input A is Output	in_a
110 - 111	(default)	No operation	alu_out = 8'b0

Design Tasks

1. Create ram.v describing the RAM per the specification above, and a memory initialization file containing your test vectors.
2. Create piso_reg.v describing the PISO per the specification above.
3. Create sipo_reg.v describing the SIPO per the specification above.
4. Create alu.v describing the ALU per the specification above.
5. Create a top-level module that instantiates ram, piso_reg, sipo_reg, and alu, wiring piso_reg's serial_out and valid to sipo_reg's serial_in and shift_en, and sipo_reg's parallel_out fields to the ALU ports as shown in the bit-field table.
6. Simulate your design using the testbench and filelist provided by your instructor.
7. Correct your design as needed.

End of Lab